

Sprint 3 — ASP.NET Core, Blazor Server & Angular 17+

Duration: Week 5 – Week 7 (3 weeks)

Curriculum: ASP.NET Core + JWT (Wk 5) | Blazor Server (Wk 6) | Angular 17 (Wk 7)

App Review 3: Viva + Project — Blazor / Angular Frontend (End of Week 7)

PS-02 Deliverable: Secured Web API + Blazor admin console + Angular patient and doctor portal — all connected and role-protected

Learning Objectives

- Configure ASP.NET Core middleware pipeline, Program.cs, and environment-based settings.
- Implement JWT authentication — generate tokens on login, validate on each API request, use claims for identity.
- Enforce role-based authorisation — [Authorize(Roles="Patient/Doctor/Admin")] protecting routes per role.
- Add global exception handling middleware — structured JSON error responses, Serilog logging wired up.
- Build Blazor Server components — lifecycle hooks, parameter passing, event callbacks, two-way binding.
- Connect a Blazor Server app to the ASP.NET Core API using HttpClient with JWT bearer tokens.
- Build Angular 17+ components, services, and reactive forms — connect to the secured API via HttpClient.
- Protect Angular routes using Route Guards (CanActivate) and attach JWT to all requests via an HTTP Interceptor.

API Endpoints

Auth: POST /api/auth/register | POST /api/auth/login | POST /api/auth/refresh-token

Patients: GET /api/patients/{id} | PUT /api/patients/{id} | GET /api/patients/{id}/health-records

Doctors: GET /api/doctors | GET /api/doctors/{id} | GET /api/doctors/{id}/availability

Appointments: GET /api/appointments | POST /api/appointments | PUT /api/appointments/{id}/status | DELETE /api/appointments/{id}

HealthRecords: GET /api/health-records/{patientId} | POST /api/health-records | GET /api/health-records/{id}

Admin: GET /api/admin/doctors | POST /api/admin/doctors | PUT /api/admin/doctors/{id} | GET /api/admin/reports/appointments

What Trainees Will Build

Three separate front-ends — Angular (Patient), Angular (Doctor), and Blazor (Admin) — all secured by JWT and calling the same ASP.NET Core Web API.

ASP.NET Core Web API — JWT Auth & Role-Based Access: Auth controller: POST /api/auth/register creates a hashed user. POST /api/auth/login verifies credentials, returns a signed JWT containing UserId, Email, and Role. POST /api/auth/refresh-token issues a new access token using a stored refresh token. All non-auth endpoints require a valid bearer token. Patient routes return 403 to Doctors and vice versa.

Global Exception Handler & Serilog: A custom exception handler catches all unhandled exceptions and returns a structured JSON error response { statusCode, message }. Stack traces are never returned to clients. Serilog is wired up to write structured logs to a rolling file sink — every request, error, and significant action is logged with contextual properties (UserId, Action, Duration).

Angular Patient Portal — Login & Registration: Login page: email + password form → POST /api/auth/login → JWT stored in localStorage. Register page: full patient details form → POST /api/auth/register → auto-login on success. HTTP Interceptor: reads JWT from localStorage and attaches it as Authorization: Bearer to every outgoing request. AuthGuard: redirects unauthenticated users to /login for all protected routes.

Angular Patient Portal — Doctor Search & Booking: Doctor Search page: search bar + specialisation filter dropdown → GET /api/doctors?specialisation={s}. Displays doctor cards with name, specialisation, experience, and fee. Book Appointment page: doctor selected → date picker (no past dates) → available slots fetched from GET /api/doctors/{id}/availability → slot selected → POST /api/appointments. Success redirects to My Appointments with a confirmation banner.

Angular Patient Portal — Appointments & Health History: My Appointments page: GET /api/appointments (filtered to logged-in patient) → list of appointment cards showing doctor, date, time, and a colour-coded status badge (Pending/Confirmed/Cancelled/Completed). Cancel button triggers PUT /api/appointments/{id}/status with a cancellation reason. Health History page: GET /api/patients/{id}/health-records → table of past visits showing visit date, doctor name, specialisation, diagnosis, and prescribed medication.

Angular Doctor Dashboard: Login with Doctor credentials — redirected to the Doctor Dashboard on login. Today's Schedule: GET /api/appointments?doctorId={id}&date=today → list of today's appointments with patient name, time slot, and current status. Patient Profile modal: clicking a patient shows their profile and health history inline. Add Health Record: after a consultation, a slide-out form captures Diagnosis, Prescription, and Notes → POST /api/health-records. The appointment status is updated to Completed simultaneously.

Blazor Server Admin Console: Separate Blazor Server application authenticated with Admin JWT. Doctor Management page: list of all doctors with Add / Edit / Toggle Active forms using Blazor components. Calls GET, POST, PUT /api/admin/doctors via HttpClient with bearer token. User Management page: view all registered users, filter by role.

Appointments Report page: GET /api/admin/reports/appointments returns daily totals — displayed as a summary table by date, with counts for Confirmed, Cancelled, and Completed.

App Review 3 — Assessment Focus

- JWT works end-to-end — register, login, token returned, token used in all subsequent API calls.
- Role-based access is enforced — Patient cannot call Doctor endpoints; returns 403 Forbidden.
- Angular HTTP Interceptor attaches JWT to every protected request automatically.
- Angular Route Guards redirect unauthenticated users to /login — no protected page accessible without a token.
- Angular app is fully component-driven — no business logic in templates, all API calls inside Angular services.
- Blazor admin console connects to the same API and all admin operations (add/edit/report) work.
- Global exception handler is active — no raw stack traces ever returned to the browser.
- Serilog logs are writing to file and include structured properties (UserId, Action).